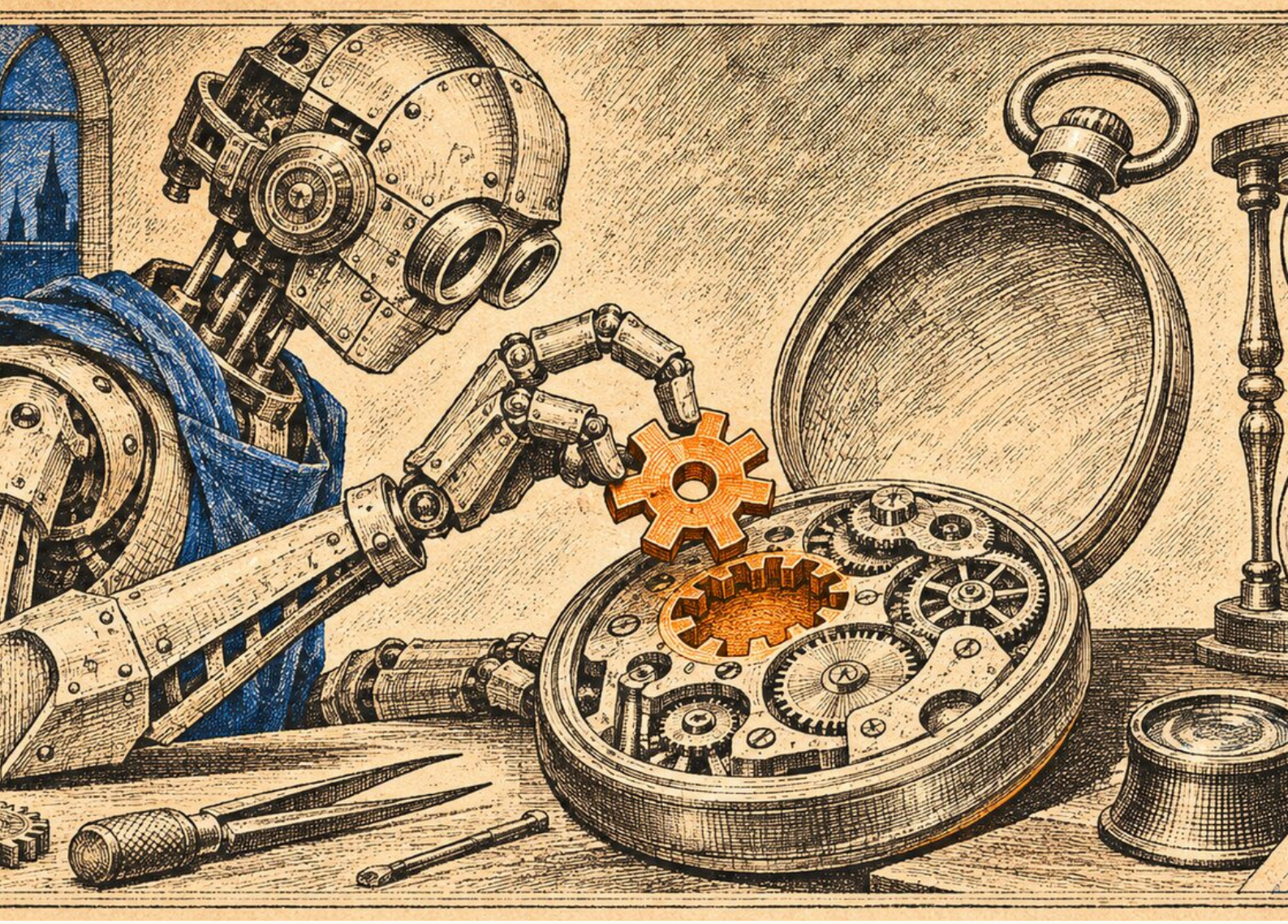


A PLAYBOOK FOR SMALL CODING AGENTS · GEMMA 4

Before You Trust the Pass Rate

Nine rules for building and evaluating an open-weight coding agent under a fixed time budget, from Gemma 4 31B on real GitHub issues. Most of them are about measuring the right thing before improving it.

72 → 102
of 129 tasks our environment could actually grade, before and after two missing test packages

31 of 32
wrong fixes passed the agent's own final check

2–3 tasks
between two runs of the same agent, on our dev set and on the leaderboard

From the paper *Fixing the Context Didn't Fix the Agent* (Koli, 2026), Gemma 4 Developer Agent Paper Track

THE PLAYBOOK ON ONE PAGE

Nine plays, each one measured

Gemma 4 31B (4-bit, 32K context) in the competition's harness: 129 public tasks from FastAPI, Rich, Requests and HTTPX, about 120 hidden tasks to finish in 12 hours. Section numbers point to the paper.

1

Audit the grader before the agent.

Run every task's reference patch and an empty patch through the grading you will use.

Only **72 of 129** tasks were gradable at first. A pilot's 37.5% was really **1 of 4**. §4

2

Check prompt + reserved output against the window.

vLLM rejects a request when the prompt plus `max_output_tokens` exceeds the context.

A 16K reservation halved a 32K window, yet replies had a median of **122** tokens. §5

3

Removing a failure is not fixing a task.

Tasks freed from one failure mode can fail in another.

Context overflows went **11 → 0**; the pass rate stayed at 42.4% vs 40.9%; wrong fixes rose from 20 to 26. §5

4

Measure noise before you claim.

Run each configuration twice and compare task flips, not percentages.

The same agent moved **13 → 15** of 33 between runs, and **14 vs 11** of 58 on the leaderboard. §3, §7

5

Label failures from the trace and the real fix.

Context, time, no submission, broken code, wrong place, wrong fix: each needs a different remedy.

46 of 48 wrong patches edited lines the real fix changed. Location was solved; the repair was not. §6

6

Don't let the agent grade itself.

Checks written from the agent's reading of the issue encode its misreading.

Final self-checks passed **31 of 32** wrong fixes; in an Agentless-style pipeline, **11 of 12**. §6

7

Keep a holdout, and believe it.

Design on dev; test once on tasks you never looked at.

A verify-first prompt gained **+4/−0** tasks on dev and **7 vs 7** on holdout. §6

8

Do the budget math first.

Total time = tasks × mean task time. Over the limit means no score at all.

At **15.7 min** per task the default setup needed ~31 h of 12; two of our submissions got no score. §7

9

Spend calls, not tokens.

Time goes to generation, and every call has a fixed cost.

A call costs about **7 s + 4 s per 100** output tokens; prompt length added nothing measurable; halving thinking changed nothing. §7

The rule behind all nine

Make sure you can **measure** a change before you **make** one, and measure it on tasks it was not designed on.

PLAY 1 IN PRACTICE

Audit the grader: gold and empty patches

Replace the agent with a fixed patch and run the harness's own grading twice per task: once with the reference fix ("gold"), once with nothing. It costs no GPU time: 25 CPU minutes for 129 tasks.

VERDICT	GOLD PATCH · EMPTY PATCH	WHAT IT MEANS FOR YOUR NUMBERS
Sound	passes · fails	Gradable. Only these tasks belong in a pass rate.
Free pass	passes · passes	Any agent "solves" it, including one that does nothing. Inflates small samples.
Unsolvable here	fails · fails	Even the real fix fails in this environment. Every agent "fails" it.

What we found, by repository (after the fixes below)

REPOSITORY	TASKS	SOUND · FREE PASS · UNSOLVABLE	CAUSE
fastapi	67		Before: 29 sound; tests import `inline-snapshot` (28) and `dirty-equals` (9)
rich	48		The reference fix fails here too (for example `test_markdown_render`)
requests	13		Tests import the image's installed `requests`, not the patched workspace
httpx	1		

Two fixes raised gradable tasks from 72 to 102: installing the two missing test packages (`inline-snapshot`, `dirty-equals`) from a wheels dataset. No fix makes the requests tasks gradable in the notebook: none of the 13 has a reference fix that changes the outcome.

Do this

- Run the audit in the exact environment your experiments use, not the one you hope the scorer uses: gradability is a property of the task *and* the environment.
- Report pass rates on sound tasks only, and say how many tasks that is.
- Re-run the audit whenever the image, the harness or the test dependencies change.
- For every unexplained gold failure, read the first failing test; it usually names the missing package.

Notebook: kaggle.com/code/rishikeshbharatkoli/gemma4-task-soundness (public), with the verdict tables and the first failing test for every unexplained failure.

PLAYS 2, 8 AND 9 IN PRACTICE

The budget math

A timed scorer turns every design choice into minutes. Work out where they go before you add anything.

```
context available for the prompt = window - max_output_tokens
  32,768 - 16,384 = 16,384  →  32,768 - 8,192 = 24,576

time per model call ≈ 7 s + 4 s × (output tokens ÷ 100)   (R² = 0.83, 3,907 calls)
run time ≈ startup + tasks × mean task time   ≤ the scorer's limit
  120 tasks in 12 h  →  about 6 minutes per task, everything included
```

What we measured

- Replies are short: median **122** output tokens, 90th percentile 718, 99th 4,142, maximum 8,401 (326 calls). An 8K reservation covers them all.
- Completions are 3% of all tokens; the rest is prompt, re-sent every call. File reads are 72–82% of tool output.
- Model generation is **98%** of agent time; commands and tests take the other 2%.
- At 8 concurrent tasks, output was **46% slower** per token than at 4. A sequential scorer is faster than your concurrent test runs.
- About 9 tasks per run hit an 8-minute cap under every prompt we tried.

What caps cost

Of the fixes our agent found with 60-minute budgets:



So an 8-minute cap gives up a measurable share of solvable tasks, and every task the capped agent solved had also been solved without caps.

Our submissions

CAPS	RESULT
60 min / 100 calls	timed out, no score
8 min / 28 calls + verify-first prompt	no score (over 12 h)
7 min / 26 calls	0.13 – 0.15
8 min / 28 calls	0.13, and 0.18 (best)

Do this: log the time per task and per call from day one, project the full run before every submission, and spend any added time on caps, not on longer prompts: a prompt that adds steps cannot help tasks that run out of time.

PLAYS 3 TO 7 IN PRACTICE

Read failures, not pass rates

Label every failed task automatically from its trace, its test log and the reference patch. The labels tell you which lever is worth pulling.

LABEL	HOW TO DETECT IT	WHAT WE FOUND
Context overflow	Context-window error in the trace	11 in the baseline's two runs; 0 after play 2
Out of time or calls	Harness timeout or call limit	About 9 per run under 8-minute caps
No submission	Empty patch	10.6% of task-runs; 1.5% after play 2
Broken code	Hidden tests crash inside the agent's new code	9 of the 48 wrong patches crashed in the new code
Wrong location	Patch touches no file the reference fix changes	3 in the baseline
Wrong fix	Right place, hidden tests fail an assertion	20 in the baseline, 26 after play 2: the largest group

What the wrong fixes looked like

- Right file in **91%** of 269 patches, though issues name it in 17% of tasks.
- Smaller than the real fix: median 17 vs 28 added lines, fixing the reported case but not its siblings.
- The test file holding the hidden tests already existed in all 48 cases; the agent ran it in 7.

Why self-checks don't help

An agent that misreads the issue writes checks that encode its misreading. Forcing verification changed behaviour as intended (a reproduction before every edit) and the checks still passed: 11 of 12 and 13 of 13 wrong fixes.

What did help, and where it stopped

Writing the issue's expected behaviour as assertions *before* editing:

TASKS	WITHOUT	WITH
Dev, 8-min caps	9, 12	14, 13
Holdout, 8-min caps	7 of 26	7 of 26

Wrong fixes fell from 14 to 8 on dev, so the mechanism is real. On holdout most failures were exhausted budgets, which a prompt that adds steps cannot fix. We designed it on dev failures, so part of its dev gain was selection.

> An independent signal of intent (the real tests, a second reading of the issue, a human) is what self-verification lacks.

PRINT THIS PAGE

Before you claim, before you submit

Each box is one of our mistakes or one of the checks that caught it.

Before you claim an improvement

- [ ] The audit ran in this environment, and the rate counts sound tasks only.
  72 of 129 at first; a pilot's 37.5% was 1 of 4.
- [ ] Each configuration ran at least twice.
  The same agent moved 2 tasks on dev and 3 on the leaderboard.
- [ ] The gain is more than about 3 tasks, or a sign test says it is not chance.
  +4/−0 had p = 0.125.
- [ ] It held on tasks you did not design on.
  +4/−0 on dev became 7 vs 7 on holdout.
- [ ] You know which failure label it moved, and where those tasks went.
  Overflows 11 → 0, wrong fixes 20 → 26.

Before you submit to a timed scorer

- [ ] prompt + `max_output_tokens` ≤ context window.
  Starter config: 16K + 16K in a 32K window.
- [ ] Projected run time fits the limit, with margin.
  Our first and sixth submissions got no score.
- [ ] You started from a configuration known to score and changed one thing.
  Three of our submissions failed before one scored.
- [ ] Added time goes to caps, not to longer prompts.
  About 9 tasks per run hit the cap under every prompt.
- [ ] You read one leaderboard score as one sample, not a verdict.
  The same agent scored 0.18 for us and 0.24 for its author.

Paper: *Fixing the Context Didn't Fix the Agent* (Koli, 2026), Gemma 4 Developer Agent Paper Track. Audit notebook: kaggle.com/code/rishikeshbharatkoli/gemma4-task-soundness. Companion: *Triage, Don't Fix* (Koli, 2026), which builds on the finding that the agent locates bugs well.